Instituto Superior de Engenharia de Lisboa

Departamento de Engenharia Eletrónica e Telecomunicações e de Computadores (DEETC)

Mestrado em Engenharia Informática e de Computadores

Module of Practice #06

Model Evaluation

Aprendizagem e Mineração de Dados (AMD)

1º Semestre – 2025/2026

Autores:

Florin Sofinet (A46608)

Diogo Guerra (A49149)

Nuno Silva (A53976)

Segunda-feira, 01 de dezembro de 2025

Índice

Introdução

1. *Orange Data Mining*

2. Carregamento de um *dataset*

3. Aplicação de métodos de separação *train-test*

4. Código completo de *train-test* e análise de resultados

5. Análise e implementação *bootstrap*

6. Construção de classificador e método de separação *train-test*

7. Classificar, *train-test* e pontuação geral

Conclusão

Introdução

Este laboratório tem como objetivo explorar métodos de avaliação de modelos de *Machine Learning*, combinando capacidades programáticas de Python com ferramentas de outras bibliotecas e do ambiente do *Orange Data Mining*.

Ao longo do trabalho, serão analisadas diversas técnicas de divisão de dados, métodos de amostragem, *bootstrap*, métricas de desempenho e construção de classificadores.

1. *Orange Data Mining*

Integramos a biblioteca pandas no ambiente virtual do Orange.

1. a)

Instalamos a biblioteca pandas no ambiente virtual do Orange.

1. b)

Verificamos as versões das bibliotecas panda, numpy e scikit-learn.

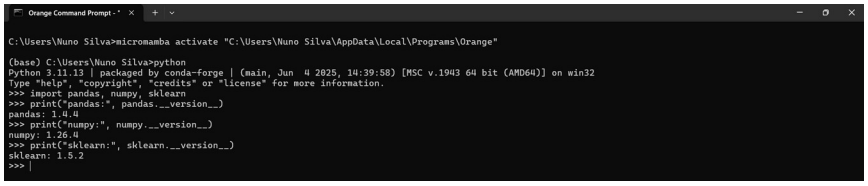


Figura 1. Verificação das versões de pandas, numpy e scikit-learn instaladas no ambiente virtual.

2. Carregamento de um *dataset*

Carregamos *datasets* para compreender como são lidos, processados e disponibilizados.

2. a)

Executamos o ficheiro my_split_and_eval.py original.

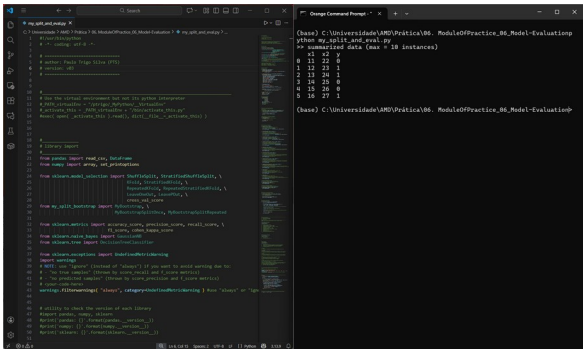


Figura 2. Execução do ficheiro my_split_and_eval.py sem alterações.

2. b)

Substituímos a função de carregamento do simple_dataset pelo carregamento direto do ficheiro pima-indians-diabetes.data.csv.

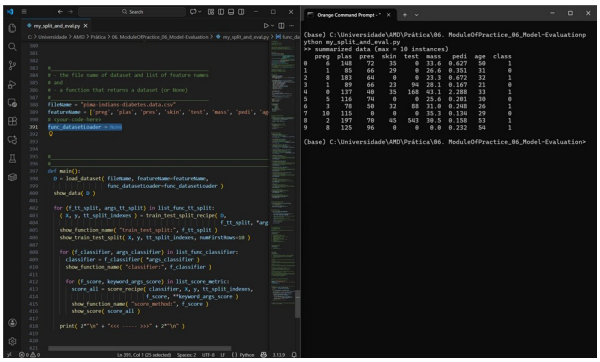


Figura 3. Substituição da função simple_dataset pelo carregamento direto do *dataset* Pima.

2. c)

Exploramos o funcionamento da função load_dataset.

2. d)

Alteramos o mecanismo de carregamento de dados para utilizar novamente simple_dataset.

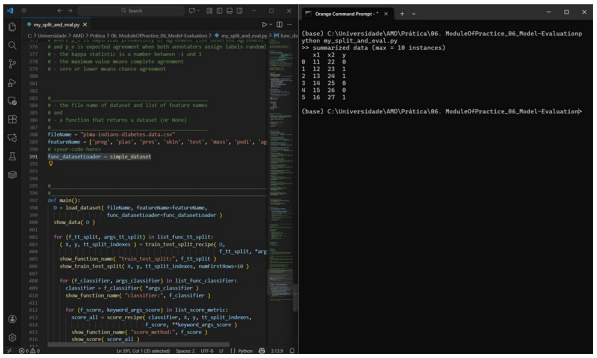


Figura 4. Carregamento do *dataset* simples através da função simple_dataset.

3. Aplicação de métodos de separação *train-test*

Aplicamos técnicas de separação *train-test*.

3. a)

Aplicamos o método *holdout* ao *dataset* simples fornecido pelo *script*.

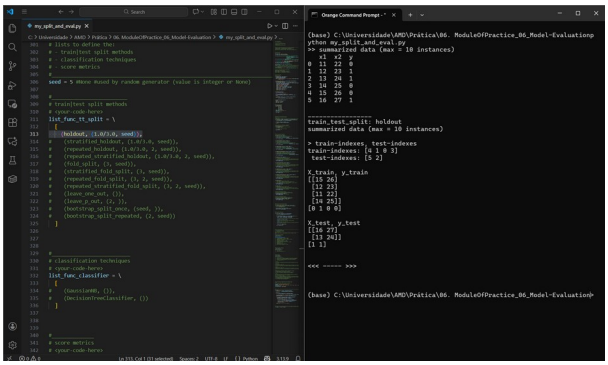


Figura 5. Aplicação do método *holdout* ao *dataset* simples.

3. b)

Repetimos o método *holdout*, alterando o rácio para 50% *train* e 50% *test*.

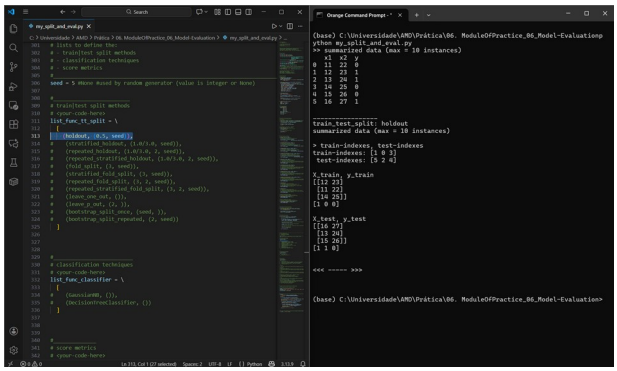


Figura 6. Método *holdout* com divisão 50/50 aplicado ao *dataset* simples.

3. c)

Aplicamos o método stratified_holdout, que preserva a proporção da classe no processo de divisão *train-test*.

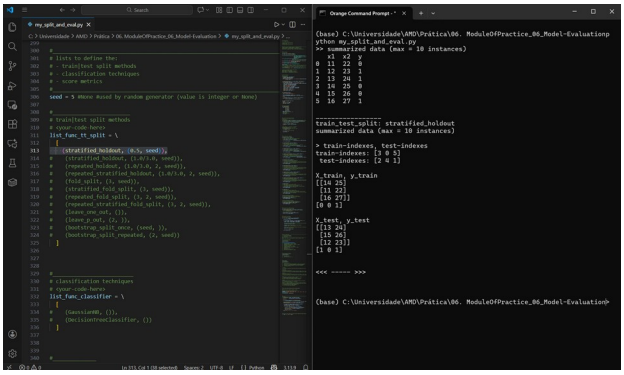


Figura 7. Divisão estratificada 50/50 preservando as proporções das classes.

3. d)

Aplicamos o método repeated_stratified_holdout, que executa várias divisões estratificadas sucessivas.

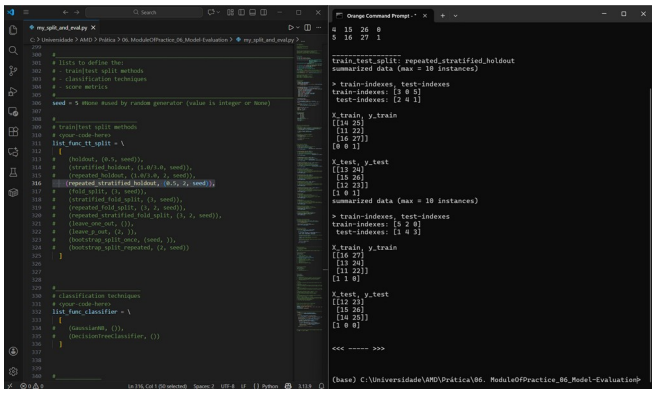


Figura 8. Aplicação do método repeated_stratified_holdout com duas repetições.

4. Código completo de *train-test* e análise de resultados

Aprofundamos as técnicas de validação, implementando vários métodos e analisando o seu comportamento.

4. a)

Todos os métodos restantes foram implementados e podem ser consultados no ficheiro *my_split_and_eval.py*.

4. b)

Os métodos implementados cobrem uma vasta gama de estratégias — desde abordagens exaustivas, como *leave_one_out e leave_p_out*, até técnicas que criam múltiplas divisões com menor custo computacional, como *fold_split e bootstrap_split_once*. Cada método serve um propósito diferente, pelo que é importante considerar qual é o mais adequado para a situação específica.

4. c)

As funções produziram resultados de classificação variados: algumas forneceram divisões precisas, enquanto outras não tiveram um desempenho tão bom. Certas abordagens revelaram-se incompatíveis com este dataset — por exemplo, *leave_one_out e leave_p_out* entraram num ciclo infinito, impedindo-nos de obter quaisquer resultados de classificação.

5. Análise e implementação *bootstrap*

Analisamos o método *bootstrap* e corrigimos a implementação incompleta fornecida.

5. a)

Usámos o IDE VSCode em vez do IDLE e obtivemos a seguinte comando:

```
AMD/PRACTICE_6 on  main [!?] via 🐍 3.9.6 via PRACTICE_6 took 7.6s
➜ python my_split_bootstrap.py


<<< ----- >>>
tt_split_indexes | once
 [([3, 5, 0, 1, 0, 4], [3, 5, 0, 1, 0, 4])]


<<< ----- >>>
tt_split_indexes | repeated
 [([3, 5, 0, 1, 0, 4], [3, 5, 0, 1, 0, 4]), ([2, 1, 3, 4, 2, 5], [2, 1, 3, 4, 2, 5]), ([4, 1, 3, 3, 4, 1], [4, 1, 3, 3, 4, 1])]
AMD/PRACTICE_6 on  main [!?] via 🐍 3.9.6 via PRACTICE_6 took 15.1s
➜
```

Figura 12. Output of running my_split_bootstrap.py

5. b)

O conjunto de teste deve conter apenas amostras não utilizadas no treino. Filtramos os índices originais para manter apenas aqueles ausentes do conjunto de treino, permitindo uma avaliação adequada do modelo.

```
AMD/PRACTICE_6 on  main [!?] via 🐍 3.9.6 via PRACTICE_6
➜ python my_split_bootstrap.py


<<< ----- >>>
tt_split_indexes | once
 [([3, 5, 0, 1, 0, 4], [2])]


<<< ----- >>>
tt_split_indexes | repeated
 [([3, 5, 0, 1, 0, 4], [2]), ([2, 1, 3, 4, 2, 5], [0]), ([4, 1, 3, 3, 4, 1], [0, 2, 5])]
```

Figura 13. Output after updating split method.

5. c)

Testámos a reamostragem bootstrap com 10, 50 e 100 repetições para demonstrar como o aumento do número de divisões treino-teste proporciona uma avaliação de modelo mais robusta através de múltiplas rondas de validação independentes.

```
AMD/PRACTICE_6 on  main [!+] via 🐍 3.9.6 via PRACTICE_6
➜ python my_split_bootstrap.py


<<< ----- >>>
tt_split_indexes | once
 [([3, 5, 0, 1, 0, 4], [2])]


<<< ----- >>>
n_repeat = 2
tt_split_indexes | repeated
 [([3, 5, 0, 1, 0, 4], [2]), ([2, 1, 3, 4, 2, 5], [0])]


<<< ----- >>>
n_repeat = 10
tt_split_indexes | repeated
 [([3, 5, 0, 1, 0, 4], [2]), ([2, 1, 3, 4, 2, 5], [0]), ([4, 1, 3, 3, 4, 1], [0, 2, 5]), ([3, 4, 1, 1, 5, 2], [0]), ([4, 5, 0, 3, 5, 1], [2]), ([1, 5, 4, 0, 1, 3], [2]), (
[1, 0, 3, 1, 5, 4], [2]), ([3, 3, 5, 1, 2, 3], [0, 4]), ([2, 0, 2, 0, 2, 4], [1, 3, 5])]


<<< ----- >>>
n_repeat = 40
tt_split_indexes | repeated
 [([3, 5, 0, 1, 0, 4], [2]), ([2, 1, 3, 4, 2, 5], [0]), ([4, 1, 3, 3, 4, 1], [0, 2, 5]), ([3, 4, 1, 1, 5, 2], [0]), ([4, 5, 0, 3, 5, 1], [2]), ([1, 5, 4, 0, 1, 3], [2]), (
[1, 0, 3, 1, 5, 4], [2]), ([3, 3, 5, 1, 2, 3], [0, 4]), ([2, 0, 2, 0, 2, 4], [1, 3, 5]), ([3, 0, 4, 1, 2, 4], [5]), ([0, 5, 4, 5, 0, 4], [1, 2, 3]), ([1, 1, 5, 1, 3, 4], [
0, 2]), ([1, 1, 0, 5, 4, 4], [2, 3]), ([2, 3, 0, 5, 1, 2], [4]), ([5, 5, 2, 0, 3, 4], [1]), ([3, 2, 4, 2, 1, 4], [0, 5]), ([1, 0, 4, 0, 0, 0], [2, 3, 5]), ([5, 4, 4, 0, 4,
 3], [1, 2]), ([3, 0, 1, 0, 5, 4], [2]), ([2, 3, 0, 1, 1, 1], [4, 5]), ([4, 2, 5, 4, 2, 5], [0, 1, 3]), ([5, 0, 1, 3, 5, 2], [4]), ([3, 0, 0, 0, 0, 5], [1, 2, 4]), ([1, 1,
 5, 4, 0, 3], [2]), ([5, 3, 4, 5, 2, 0], [1]), ([5, 5, 5, 5, 4, 4], [0, 1, 2, 3]), ([2, 0, 2, 2, 4, 4], [1, 3, 5]), ([3, 5, 4, 0, 3, 4], [1, 2]), ([4, 0, 2, 2, 1, 1], [3,
5]), ([1, 2, 2, 1, 5, 4], [0, 3]), ([1, 4, 1, 4, 0, 1], [2, 3, 5]), ([5, 2, 0, 1, 2, 2], [3, 4]), ([3, 4, 3, 4, 5, 2], [0, 1]), ([1, 3, 5, 3, 4, 4], [0, 2]), ([1, 1, 1, 0,
 4, 3], [2, 5]), ([3, 5, 0, 0, 2, 1], [4]), ([0, 3, 4, 2, 0, 1], [5]), ([3, 4, 2, 4, 4, 1], [0, 5]), ([4, 0, 1, 5, 2, 0], [3]), ([4, 3, 1, 5, 3, 3], [0, 2])]
```

Figura 14. Output with different repetitions.

5. e)

Implementámos métodos de divisão treino-teste bootstrap criando instâncias das classes MyBootstrapSplitOnce e MyBootstrapSplitRepeated já importadas no topo do ficheiro, permitindo a reamostragem bootstrap única e repetida para a avaliação de modelos.

6. Construção de classificador e método de separação *train-test*

Implementados os métodos de separação, construímos um classificador real.

6. a)

O Exercício 6 requer a implementação de um classificador básico utilizando o método de divisão treino-teste holdout. O primeiro passo exigiu comentar todos os métodos de divisão treino-teste na lista *list_func_tt_split*, exceto o método holdout. Isto permite-nos focar num único método de divisão e verificar a sua implementação corretamente

6. b)

O exercício solicita procurar a tag <your-code-here> e aplicar o DecisionTreeClassifier.

Ao executar o código inicial, observámos “ embora o classificador tenha sido criado, nenhum método de divisão treino-teste estava ativo para treinar e avaliar o modelo.

Na lista de classificadores (linhas 354-358), ativamos o DecisionTreeClassifier:

```
1 list_func_classifier = \
2   [
3 #     (GaussianNB, ()),
4     (DecisionTreeClassifier, ())
5   ]
```

Esta configuração é utilizada na função main() (linha 409) onde o classificador é instanciado:

```
1 classifier = f_classifier( *args_classifier )
```

Sem um método de divisão treino-teste ativo na lista *list_func_tt_split*, o classificador não é treinado nem avaliado, resultando em nenhuma saída visível no terminal.

6. c)

O próximo passo foi ativar o método de divisão treino-teste holdout (utilizando DecisionTreeClassifier).

Ao executar o código, observámos que agora é necessário aplicar a função *score_recipe* utilizando algum(ns) método(s) de pontuação para avaliar o desempenho do modelo.

Na lista de métodos de divisão (linhas 328-341), ativamos o método holdout:

```
list_func_tt_split = \
  [
   (holdout, (1.0/3.0, seed)),
#    (stratified_holdout, (1.0/3.0, seed)),
#    (repeated_holdout, (1.0/3.0, 2, seed)),
#    ...
  ]
```

Isto configura uma divisão holdout com 1/3 dos dados para teste (aproximadamente 33%) e 2/3 para treino (aproximadamente 67%), utilizando seed=5 para reprodutibilidade.

Ao executar o script com apenas o método holdout ativo, obtivemos:

Métrica	Valor
Exatidão (Accuracy)	66.15%

Interpretação dos Resultados:

A exatidão de 66.15% significa que o classificador prevê corretamente a classe para aproximadamente 2 em cada 3 instâncias

Este valor está acima do baseline de classe maioritária (65.1%), indicando que o modelo aprendeu alguns padrões.

7. Classificar, *train-test* e pontuação geral

Aplicamos o classificador, classificamos métricas e fazemos uma interpretação geral. Analisamos, finalmente, a gestão dos *warnings*.

7. d)

Procurámos a tag <your-code-here> na função score recipe (linhas 296-314) e completámos o código:

```
# predict using the model and X_test (testing dataset)
y_predict = classifier.predict( X_test )

# score the model using y_test (expected) and y_predict (
   predicted by model)
score = f_score( y_test, y_predict, **keyword_args_score )
```

7. e)

Testámos o código com a configuração especificada: repeated holdout(1.0/3.0, 2, seed) e DecisionTreeClassifier com seed=5.

Resultados obtidos:

```
score_method: accuracy_score
::all-evaluated-datasets::
50.00% | 0.00% |
25.00% (+/- 25.00%)

score_method: precision_score
::all-evaluated-datasets::
100.00% | 0.00% |
50.00% (+/- 50.00%)

score_method: recall_score
::all-evaluated-datasets::
50.00% | 0.00% |
25.00% (+/- 25.00%)

score_method: f1_score
::all-evaluated-datasets::
66.67% | 0.00% |
33.33% (+/- 33.33%)

score_method: cohen_kappa_score
::all-evaluated-datasets::
0.00% | 0.00% |
0.00% (+/- 0.00%)
```

7. f)

Durante a execução, surgiram avisos (warnings). Identificámos os seguintes textos importantes:

Aviso ”no predicted samples”:

```
UndefinedMetricWarning: Precision is ill-defined and being set to
    0.0
due to no predicted samples.
```

Aviso ”no true samples”:

```
UndefinedMetricWarning: Recall is ill-defined and being set to
   0.0
due to no true samples.
```

7. g)

”no predicted samples”:

- Significa que o classificador não previu nenhuma instância como positiva (TP + FP = 0)
- Todas as previsões foram da classe negativa
- Afeta a métrica de precisão (precision)

”no true samples”:

- Significa que o conjunto de teste não contém nenhuma instância positiva verdadeira (TP + FN = 0)
- Todas as instâncias reais no conjunto de teste são da classe negativa
- Afeta a métrica de revocação (recall)

Aviso: no true samples (recall e f1-score)

A revocação (recall) é definida como:

$$\text{Recall} = \frac{TP}{TP + FN}$$

Quando não há amostras verdadeiras positivas no conjunto de teste, temos TP + FN = 0, o que resulta em divisão por zero. O F1-score depende de recall, portanto herda este problema.

A precisão (precision) é definida como:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Quando o classificador não prevê nenhuma amostra como positiva, temos TP + FP = 0, o que resulta em divisão por zero. O F1-score depende de precision, portanto herda este problema.

7. h)

Lemos a documentação do scikit-learn para as funções precision score e recall score. Utilizámos o parâmetro zero division para eliminar os avisos.

Na lista de métricas (linhas 365-373):

```
list_score_metric = \
  [
   (accuracy_score, {}),
   (precision_score, {"average":"weighted", "zero_division":0}),
   (recall_score, {"average":"weighted", "zero_division":0}),
   (f1_score, {"average":"weighted", "zero_division":0}),
   (cohen_kappa_score, {}),
  ]
```

O parâmetro zero division=0 define o valor da métrica como 0.0 quando ocorre divisão por zero, eliminando os avisos.

7. i)

Procurámos a tag <your-code-here> e utilizámos o valor ”ignore” para desativar a apresentação de avisos da categoria UndefinedMetricWarning.

No início do ficheiro (linhas 39-43):

```
from sklearn.exceptions import UndefinedMetricWarning
import warnings

warnings.filterwarnings( "ignore", category=
   UndefinedMetricWarning )
```

Aviso importante: Ao desativar estes avisos, perde-se informação importante sobre problemas no conjunto de dados ou no modelo. Só se deve fazer isto quando realmente se conhece o impacto correspondente!

Conclusão

A realização deste laboratório permitiu percorrer todas as etapas de avaliação de modelos preditivos: preparação do ambiente, carregamento de dados, divisão correta de *test-train*, implementação de métodos avançados de amostragem e integração de métricas de desempenho.

No final, o laboratório fornece uma visão completa das práticas recomendadas para avaliar modelos de *Machine Learning* de forma rigorosa.